

Auths-Proof: Mechanically Refining Rich Authorization Semantics to Shipping Rust

bordumb · bordumbb@gmail.com

29 July 2026

ABSTRACT

Authorization systems are often verified at the wrong boundary. A proof assistant may establish elegant properties of a model while production code independently reimplements the model, leaving semantic correspondence to review and testing. We present the formal core of **Auths-Proof**, a deterministic proof-carrying authorization kernel, and a refinement chain that connects its rich authority semantics to the exact safe Rust functions used in production.

Auths-Proof models permissions and audiences as finite sets, validity as inclusive intervals, action authority as a closed constraint algebra, budgets as optional ceilings, status as an ordered freshness policy, profile selection as a state transition, and delegation as a linked, strictly depth-decreasing chain. Lean 4 proves the component laws, semantic containment, downward-closed action coverage and evidence requirements, unique accepted transitions, ordered diagnostics, and three-valued composition properties. Production authoring, delegation, and terminal coverage were reshaped into total, pure, unsafe-free Rust evaluators over validated borrowed views. Charon and Aeneas translate those exact functions into Lean; three refinement theorems prove agreement with the handwritten rich specification.

The evaluated artifact inventories 72 compiled Lean statements: 56 rich authority and production-refinement claims and 16 composition claims. It supplements the proofs with 23 rich semantic vectors, a required 21-mutation matrix, all 1,024 Boolean attenuation projections, 2,448 threshold states through the default 16-leaf deployment bound, and four Kani harnesses. Statement hashes, transitive axioms, translation source closure, pinned tools, external models, and generated artifacts are checked in a read-only repository gate.

This is not a proof of the complete verifier. Canonical decoding, cryptography, evidence acquisition, registries, clocks, stores, adapters, credentials, external effects, the Rust compiler, and the reviewed representation-validity premises remain outside or inside an explicit trusted computing base. The result is stronger than two matching implementations and narrower than whole-program verification: production authority decisions are mechanically represented and proved against a readable rich model, while the remaining assumptions stay inspectable.

1. Introduction

An authorization verifier answers a deceptively small question:

Does authority trusted by this verifier authorize this exact action under this exact context?

The answer is security-critical because identity is not authority. A valid signature establishes control of a key. A WebAuthn ceremony establishes control of a credential under ceremony conditions. A certificate establishes a path to a trust root. None of those facts alone grants permission to deploy software, spend a budget, invoke a tool, or mutate a repository. Authentication logics and authorization logics have long treated these as different judgments (Burrows, Abadi, and Needham 1990; Abadi et al. 1993). Capability and trust-management systems likewise make authority and delegation explicit (Dennis and Horn 1966; Blaze, Feigenbaum, and Lacy 1996; Blaze et al. 1999).

Auths-Proof implements this separation as an offline kernel:

$$\text{verify}(P, A, C) \rightarrow \text{Authorized}(S) \mid \text{Denied}(d) \mid \text{Indeterminate}(q).$$

P is a portable proof graph, A is a profile-canonical action, and C is verifier-trusted context. The kernel performs no network, clock, filesystem, database, or key-custody I/O. Only **Authorized** contains a sealed action value eligible for execution. **Denied** records a stable permanent failure. **Indeterminate** records a stable missing trustworthy fact. The latter two outcomes never permit execution.

This paper is about a narrower problem than the full protocol: how do we know that the algebra proved in Lean is the algebra executed by shipping Rust?

Writing the same function twice does not answer that question. Tests over a few examples do not answer it. A shared trait name does not answer it. Even a machine-checked proof can create false confidence when its definitions are detached from production. Experience from verified compilers and kernels shows that useful assurance depends on an explicit refinement chain and an explicit trusted computing base (Leroy 2009; Klein et al. 2009).

Auths-Proof therefore treats linkage as a first-class artifact at two boundaries. A small versioned contract generates the finite Boolean algebra surface in Rust and Lean. Separately, Charon and Aeneas translate the exact pure Rust authority evaluators and their leaf predicates into Lean. Handwritten theorems refine the translated functions to a readable rich specification. Compiled-statement auditing, semantic vectors, mutation witnesses, and Kani provide independent change detectors around that proof chain. Figure 1 summarizes the resulting evidence graph.

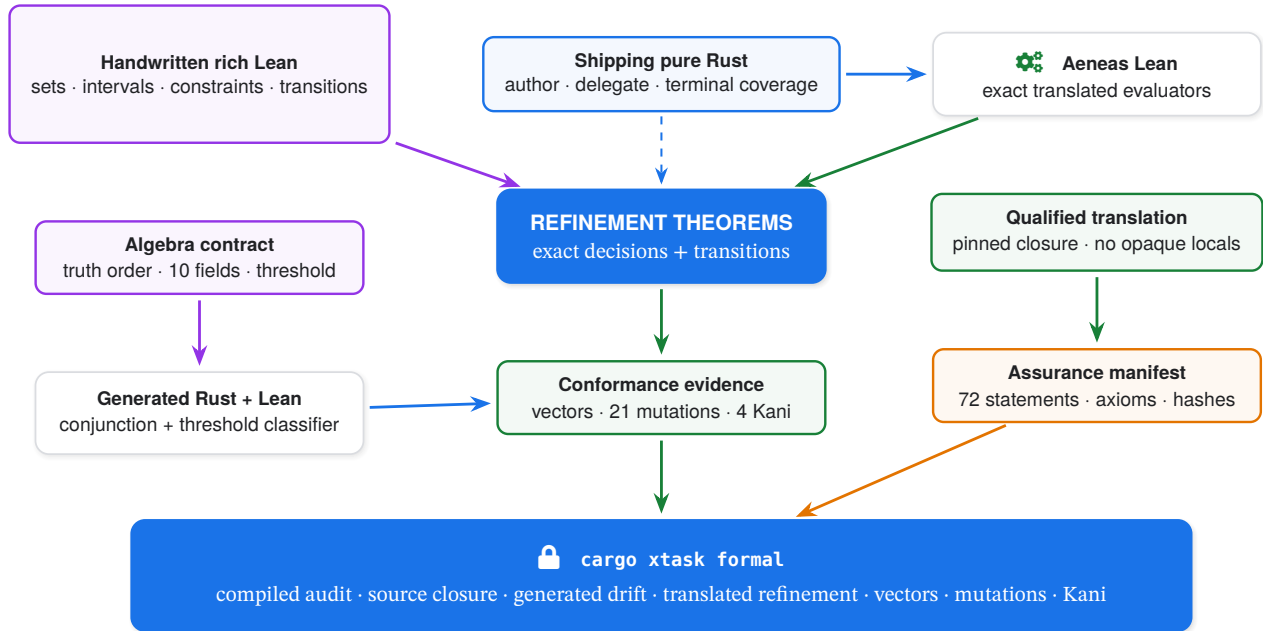


Figure 1: The production refinement chain. The finite algebra remains generated from one contract. The rich authority path translates shipping Rust, then proves its decisions against a handwritten semantic model. Both paths converge in one reproducible gate.

1.1 Contributions

This paper makes five contributions.

A closed authorization algebra. Delegation combines a product order over profile, permissions, validity, audiences, action constraint, budget, status, and assurance with exact root linkage and strictly decreasing depth. Composition is a three-valued algebra that preserves trustworthy uncertainty.

A rich, unbounded Lean model. The Lean development represents finite permission and audience sets, inclusive windows, action constraints, optional budgets, status freshness, profile selection, chain linkage, and terminal action membership directly. It proves order, coverage, evidence, delegation-depth, accepted-transition, diagnostic, and composition properties.

A generated language boundary. A declarative TOML contract generates the Rust trait and functions and the corresponding Lean structure and functions. Changing a dimension is a cross-language schema change rather than two independent edits.

A mechanical rich-authority refinement. Shipping code calls pure production evaluators that Charon and Aeneas translate into Lean. Refinement theorems cover pre-signing scope decisions, delegation linkage and its unique accepted transition, terminal action coverage, and each caller’s stable first-failure result under validated-representation premises.

An executable assurance ledger. A 72-claim manifest binds English claims to exact compiled theorem statements,

hashes, source closures, Rust symbols, evidence, transitive axioms, toolchain locks, and residual assumptions. Rich vectors, a required mutation matrix, finite exhaustive vectors, and Kani harnesses supplement rather than substitute for the proofs.

1.2 Scope and terminology

The paper uses *proof* in three different senses:

- an **authorization proof** is untrusted protocol input;
- a **Lean proof** is a kernel-checked theorem;
- a **Kani proof harness** is a bounded symbolic program check.

These are not interchangeable. The formal model excludes parsing, canonical CBOR correctness, cryptographic soundness, principal adapters, graph resolution, clocks, evidence acquisition, durable stores, external effects, and complete verifier control flow. It does include rich authority semantics and the isolated production decisions that author, delegate, and cover terminal actions. Those conclusions hold over constructor-validated Rust views satisfying explicitly stated representation invariants.

2. Authorization semantics

2.1 Trusted context and portable evidence

The portable proof carries signed grants, signed actions, evidence objects, bindings, and an authorization plan. It does not carry verifier trust. Trust anchors, accepted registries, evaluation time, expected challenge and audience, status and assurance policy, composition floors, and resource limits enter through *C*.

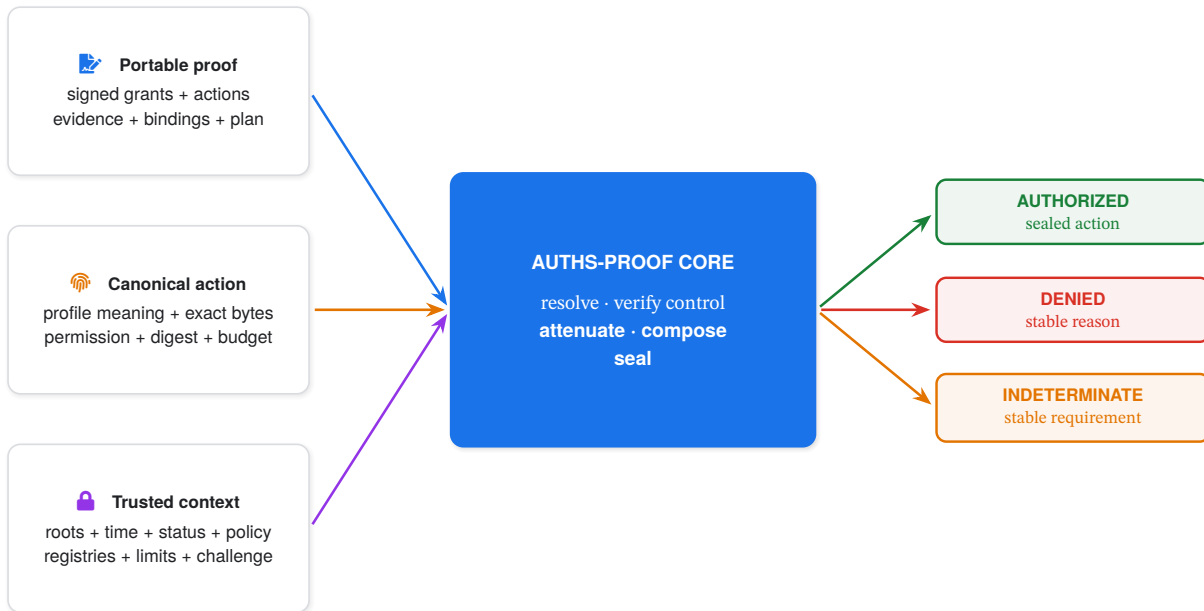


Figure 2: The verifier’s narrow waist. Identity evidence contributes facts, but only the local authority computation can produce a sealed action.

This partition instantiates verifier sovereignty. A proof can demonstrate a chain from a root, but it cannot choose the local root. It can carry a status statement, but it cannot choose the accepted status method or freshness limit. It can request a composition plan, but the verifier can impose additional branch, actor, and root diversity floors.

2.2 Three-valued truth

Each proof branch produces one value in

$$\mathbb{T} = \{\perp, ?, \top\}, \quad \perp \leq ? \leq \top,$$

where $\top$ is authorized, $\perp$ is denied, and $?$ is indeterminate. Indeterminate is not an error code disguised as authority. It means that a recognized trustworthy fact is absent or unavailable and that authorization would still be reachable if that fact were supplied.

For conjunction and disjunction:

$$x \wedge y = \min_{\leq}(x, y), \quad x \vee y = \max_{\leq}(x, y).$$

The implementation evaluates all members in canonical order so diagnostic selection remains deterministic even when the truth algebra is permutation invariant.

For a threshold requiring k successes, let a be the number of authorized branches and u the number of indeterminate branches:

$$\text{threshold}(k, a, u) = \begin{cases} \top & a \geq k, \\ ? & a < k \wedge a + u \geq k, \\ \perp & a + u < k. \end{cases}$$

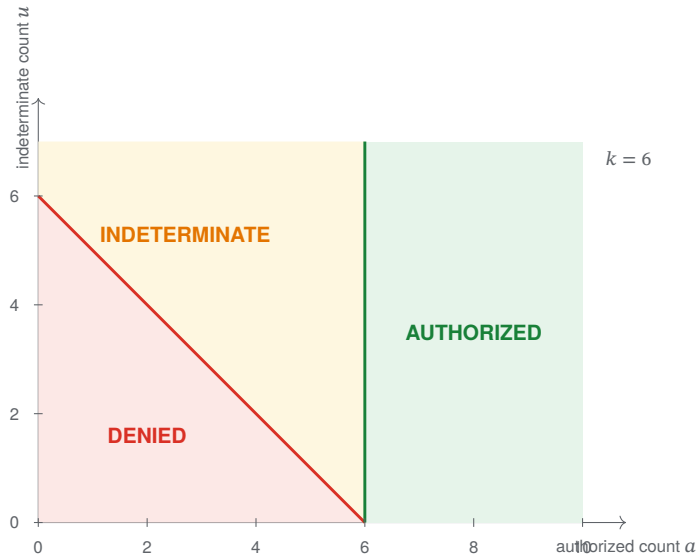


Figure 3: Threshold partition for $k = 6$. The three regions are total and mutually exclusive. Increasing k moves the authorization boundary rightward and cannot create authority.

Preserving $?$ matters operationally. Collapsing it into $\perp$ loses the difference between “the statement is invalid” and “a required status snapshot is unavailable.” Collapsing it into $\top$ fails open.

2.3 Delegation as a product order

The rich model separates semantic scope from chain position. An authority scope is:

$$S = (p, \pi, v, a, c, b, t, h),$$

where p is profile-selection state, π a finite permission set, v an inclusive validity interval, a a finite audience set, c an action constraint, b an optional budget ceiling, t a status policy, and h an assurance identifier. A chain state is:

$$E = (r, s, S, d, g),$$

with local root r , current subject s , scope S , remaining delegation depth d , and optional last-grant identifier g . Subject, depth, and last grant are transition state; treating them as ordinary partially ordered authority would make false antisymmetry claims.

For child scope S' and parent scope S :

$$S' \sqsubseteq S \iff p' \leq p \wedge \pi' \subseteq \pi \wedge v' \leq v \wedge a' \subseteq a \\ \wedge c' \leq c \wedge b' \leq b \wedge t' \leq t \wedge h' = h.$$

A valid grant transition is stricter than scope containment:

$$\text{delegates}(E, G, E') \Rightarrow S' \sqsubseteq S \wedge r' = r \wedge d' < d,$$

and additionally requires exact issuer/parent linkage, selects or preserves the profile, updates the subject and last-grant identifier, and constructs the unique accepted next state. Permission and audience actions use membership, not subset. Validity uses interval containment. Action constraints order AnyBody, allowed digest sets, and exact digests. None is the unbounded authority budget, while an absent action budget request means no requested spend and is covered by every ceiling. Snapshot status preserves the method and can only demand fresher evidence. Assurance is invariant.

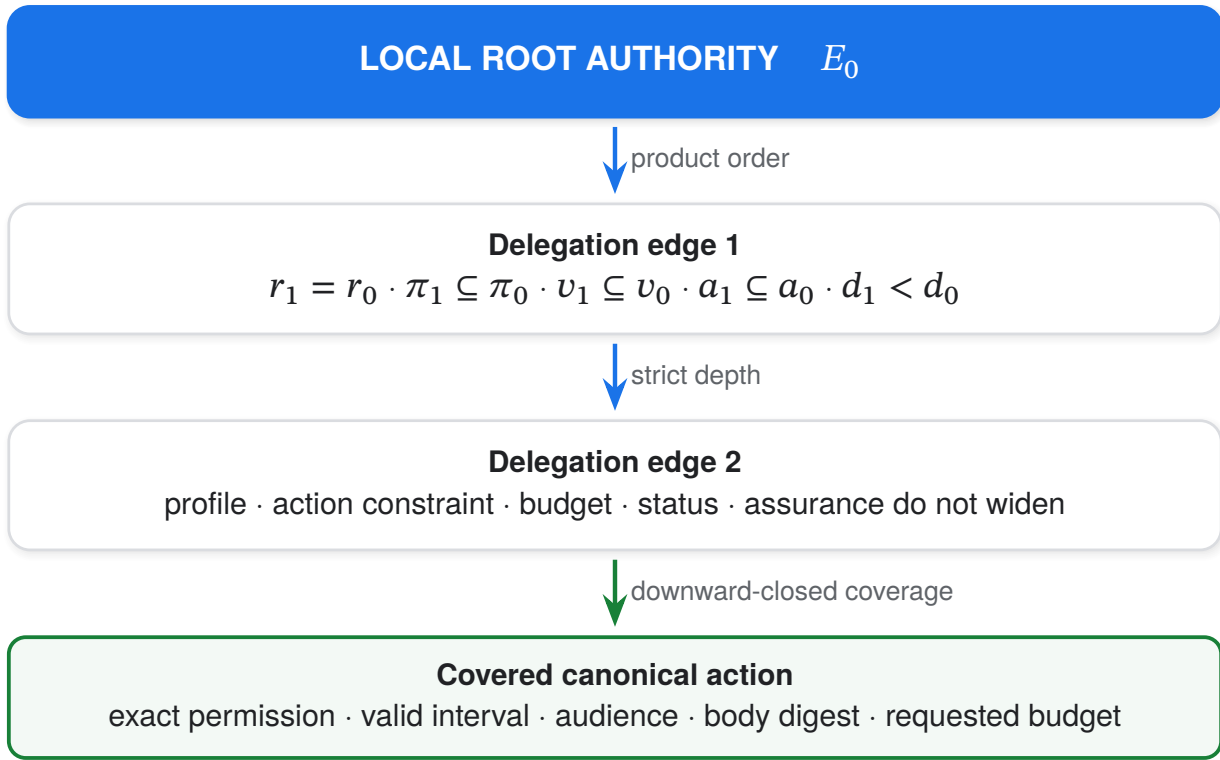


Figure 4: Authority narrows along a chain. If the terminal authority covers an action, every ancestor also covers it; the converse is intentionally false. Strictly decreasing depth bounds chain length.

3. Lean specification

3.1 Rich semantic values

Lean 4 combines an interactive theorem prover with an executable functional language and a small proof-checking kernel (Moura and Ullrich 2021). Auths-Proof uses opaque carrier types for identities and Mathlib `Finset` values for extensional sets. Numeric ordering is used only where the protocol itself is numeric: timestamps, freshness ages, ceilings, and remaining depth. No permission, audience, profile, principal, digest, or algebra identifier is ordered by an arbitrary integer encoding.

The central types are intentionally independent of Rust allocation and wire layout:

```

structure AuthorityScope (v : Vocabulary) where
  profileScope      : ProfileScope v

```

```

permissions      : FiniteSet (Permission v)
validity          : InclusiveWindow
audiences        : FiniteSet (Audience v)
actionConstraint : ActionConstraint v
budget           : Option (BudgetCeiling v)
status           : StatusPolicy v
assurance        : AssurancePolicy v

```

```

structure ChainState (v : Vocabulary) where
  root      : Principal v
  subject   : Principal v
  scope     : AuthorityScope v
  remainingDepth : Nat
  lastGrant : Option (GrantId v)

```

The model defines two related notions. `structuralScopeLe` is the decidable target-V1 relation executed by the authority kernel. `semanticAttenuates` states the denotational safety property:

```

def semanticAttenuates (child parent : AuthorityScope v) : Prop :=
  ∀ facts, admits child facts → admits parent facts

```

`admits` combines action coverage with already-validated status and assurance facts. It performs no I/O and makes no claim that external evidence is true. The proof `structural_scope_le_implies_semantic_attenuation` connects the efficient structural decision to semantic containment.

3.2 Component and containment theorems

The rich development proves reflexivity, transitivity, canonical antisymmetry, and the appropriate coverage bridge for every ordered component:

- finite-set subset and membership monotonicity for permissions and audiences;
- inclusive-window containment and action-window monotonicity;
- the complete `AnyBody` / `AllowedBodyDigests` / `ExactBodyDigest` relation;
- optional-budget ordering with `None` as unbounded authority;
- requested-budget coverage, including the distinct no-request case;
- expiry-only and method-preserving snapshot freshness policies; and
- profile selection from the root set followed by exact profile preservation.

These component lemmas compose into three security statements:

$$\begin{aligned}
 S' &\sqsubseteq S \wedge \text{covers}(S', A) \Rightarrow \text{covers}(S, A), \\
 S' &\sqsubseteq S \wedge \text{evidenceOK}(S', F) \Rightarrow \text{evidenceOK}(S, F), \\
 S' &\sqsubseteq S \wedge \text{admits}(S', (A, F)) \Rightarrow \text{admits}(S, (A, F)).
 \end{aligned}$$

This is the required safety direction: narrowing cannot create a descendant authorization that an ancestor's scope would reject. The theorem is about complete semantic facts; acquiring and validating those facts remains a separate verifier responsibility.

3.3 Grant transitions and deterministic diagnostics

A grant is accepted only when issuer and parent identifiers link to the current chain state, every scope dimension attenuates, and remaining depth strictly decreases. Lean constructs the next state from the accepted grant rather than allowing an arbitrary child state:

```

def delegates (parent : ChainState v) (grantId : GrantId v)
  (grant : Grant v) (child : ChainState v) : Prop :=
  linked parent grant ∧
  ∃ checks : scopeDepthChecks parent grant,
    child = acceptedNextState parent grantId grant checks

```

The development proves root preservation, exact subject and last-grant update, strict depth, non-widening scope, unique accepted state, transitive attenuation, and a chain-length bound:

$$\text{length}(\text{delegation successors}) \leq \text{remainingDepth}(\text{start}).$$

It also specifies the ordered diagnostic functions used by authoring, delegation, and terminal coverage. Acceptance is proved equivalent to the logical predicate; every denial is proved unsound for authorization; and delegation's first failure is characterized as broken linkage before aggregate scope expansion. Diagnostics report a result but never influence the truth of the authorization judgment.

The generated ten-Boolean boundary is retained. Lean proves that its conjunction accepts exactly the rich scope-and-depth checks. Linkage and unique next-state construction are proved separately, avoiding the false claim that the Boolean projection alone represents a complete grant transition.

3.4 Composition theorems

For both `all` and `any`, Lean proves commutativity, associativity, and idempotence. It proves that one-of-two is `any`, two-of-two is `all`, and that tightening a threshold cannot increase truth:

$$\text{thresholdTwo}(2, x, y) \leq \text{thresholdTwo}(1, x, y).$$

The unbounded count classifier has three soundness theorems:

$$\begin{aligned} \text{threshold}(k, a, u) = \top &\Rightarrow k \leq a, \\ \text{threshold}(k, a, u) = \perp &\Rightarrow a + u < k, \\ \text{threshold}(k, a, u) = ? &\Rightarrow a < k \leq a + u. \end{aligned}$$

Plan traversal is modeled structurally. Lean proves that the defined visit list is exactly the leaf list, that every finite plan has a node count, and that the declared structural cost equals that count. These are algebra-level results, not a cost proof for cryptography or allocation.

Composition floors are ordered componentwise over authorized branches, distinct actors, and distinct roots. The model proves that a tighter satisfied floor implies every looser floor is also satisfied. Thus raising local diversity requirements cannot create authorization.

3.5 Proof inventory

Table 1 groups the 72 exact declarations in the assurance manifest. The repository audits these names and statements from the compiled Lean environment, computes their transitive axioms, and rejects `sorryAx`, renamed claims, changed statements, and unreviewed additions. All 72 are marked `proved`; the audit does not infer that an English claim matches a proposition, so each manifest entry also records formal review and residual assumptions.

Family	Representative obligations	Count
Rich component relations	sets, intervals, constraints, budgets, status, profiles, and monotonicity bridges	24
Scope and semantic containment	scope order, canonical antisymmetry, action/evidence/admission closure, decidability	11
Delegation, coverage, diagnostics	linkage, unique transition, root, depth, chains, projection, ordered decisions	18
Production refinement	translated author-scope, delegation, and terminal-coverage evaluators	3
Composition baseline	three-valued semilattice, threshold, swap, traversal, cost, and soundness	16
Manifest total		72

Table 1: Compiled assurance inventory. Counts describe exact reviewed declarations, not proof difficulty or whole-system coverage.

4. From model to shipping code

4.1 The correspondence problem

A traditional refinement approach can prove that an implementation simulates an abstract specification. CompCert and seL4 demonstrate the strength of such end-to-end refinement arguments (Leroy 2009; Klein et al. 2009). Translation validation instead validates the result of a particular translation rather than proving a translator correct for all inputs (Pnueli, Siegel, and Singerman 1998).

The first Auths-Proof artifact linked only the final conjunction of ten Boolean attenuation checks. That proved aggregation after the security-relevant work had already happened. Production still computed those Booleans with rich Rust relations such as sorted-set subset, interval containment, action-constraint matching, and optional-budget ordering. Rewriting equivalent `Finset` definitions in Lean would have produced a better specification but not a proof that Rust implemented it.

Auths-Proof therefore uses a narrower extraction boundary than the complete verifier and a richer boundary than the Boolean conjunction. Production authority code was reshaped into total, pure, safe Rust functions; Charon lowers those exact functions to LLBC; Aeneas translates them to Lean (Ho and Protzenko 2022); and handwritten proofs connect the translated functions to the rich model. No verification-only Rust implementation is maintained.

4.2 Production reshape

The shipping crates now expose three pure authority decisions over borrowed, lossless views:

```
evaluate_author_scope_view(parent, child)
evaluate_grant_view(parent, grant_id, grant)
evaluate_action_coverage_view(authority, action)
```

`auths-model` owns the leaf predicates: profile and principal equality, permission/audience/digest membership and subset, inclusive-window containment, action-constraint allowance and attenuation, optional-budget attenuation and coverage, status-policy attenuation, and assurance equality. `evaluate_grant_view` owns linkage, all ten attenuation checks, aggregate acceptance, stable denial choice, and the fields of the accepted transition. `evaluate_action_coverage_view` owns terminal linkage, membership, containment, requested-budget coverage, and ordered denials.

Shipping authoring, `EffectiveAuthority::delegate`, and `EffectiveAuthority::authorizes` call these functions. The mutable delegation wrapper only clones the accepted fields into state; it does not recompute the decision. The private root and assurance policy are retained by that wrapper. This structural commit is deliberately distinguished from the translated decision theorem.

4.3 Qualified Charon/Aeneas translation

The translation environment pins shipping Rust 1.97.1, extraction nightly 2026-06-01, Charon 0.1.225 at a specific commit, Aeneas at a specific commit, Lean 4.31.0, and Kani 0.67.0. The qualification translates:

- 42 local `auths-model` functions with one transparent standard-library external;
- the generated attenuation conjunction;
- four `auths-authority` functions with 16 transparent links to separately translated model/algebra functions; and
- no opaque local function.

The only standard-library semantic model is `String::as_bytes`, represented as the exact UTF-8 bytes on Auths' validated bounded-string domain and failure outside the Aeneas carrier bound. Authority carrier links and leaf-function links import separately translated definitions under exact generated names; they are not axioms. The qualification requires zero compiled external axioms for these definitions, inventories every template axiom and upstream warning, reproduces the translation twice, and requires byte-identical generated output.

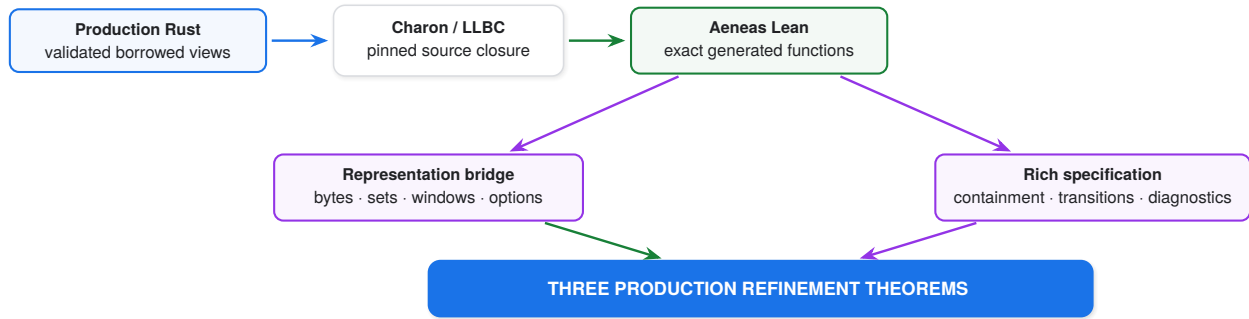


Figure 5: Rich production refinement. Translation preserves the exact production evaluator. Proofs relate translated carriers and decisions to the handwritten authority semantics under explicit representation premises.

4.4 Representation bridge

Aeneas erases private Rust newtype constructors into Lean carriers. The refinement is therefore stated for values satisfying the invariants enforced by shipping constructors:

- strings are non-empty and protocol-bounded;
- permission and audience collections are non-empty, bounded, sorted, and duplicate-free;
- profile sets are bounded and selected profiles belong to the retained root set;
- validity windows are well formed;
- snapshot freshness limits are non-zero; and
- fixed-width numeric values embed exactly into Lean naturals.

The proof layer defines abstraction functions from translated Rust values to rich identities, `Finset` scopes, windows, constraints, budgets, status, and actions. It proves that byte equality matches rich atom equality, binary-search membership and subset match extensional finite-set relations, interval comparisons match inclusive containment, and each policy predicate matches its rich relation.

This closes the validated-model-to-rich-semantics gap. It does not prove that arbitrary bytes decode into valid values, or that signatures and evidence are sound. Canonical decoding, constructor enforcement, cryptography, and the view's losslessness remain separately tested or assumed boundaries.

4.5 Production refinement theorems

Three top-level theorems cover distinct public decisions.

`translated_rust_refines_rich_spec` proves that the translated pre-signing scope evaluator returns exactly the rich target-V1 result and first failing authority dimension.

`translated_coverage_refines_rich_spec` proves that terminal coverage agrees with the rich ordered decision, including actor and grant linkage, exact profile behavior, permission and audience membership, interval containment, action constraints, and the distinct requested-budget semantics.

`translated_delegation_refines_rich_spec` proves that the translated grant evaluator agrees with rich linkage and attenuation and returns the exact accepted transition fields. Root and assurance retention occur in the structural caller around that returned transition and remain named as a residual boundary rather than being silently folded into the theorem.

Each theorem carries representation-validity premises. None assumes the semantic result of a Rust leaf predicate; those leaf predicates are themselves translated and refined.

4.6 The generated finite algebra remains

The source for the shared Boolean aggregation and threshold classifier remains `formal/algebra-contract-v1.toml`. It declares the three-valued truth order, ten named attenuation fields, conjunction acceptance, a threshold partition, and the default exhaustive bound of 16. The generator deterministically renders:

- `core/crates/auths-algebra-kernel/src/generated.rs`; and
- `formal/Auths/Generated/Algebra.lean`.

Normal verification compares both files byte-for-byte with fresh renderings. Intentional changes require `cargo xtask`

formal --update. The generated boundary still prevents silent dimension drift and supplies a compact exhaustive regression surface; it is no longer the sole connection between production authority and Lean.

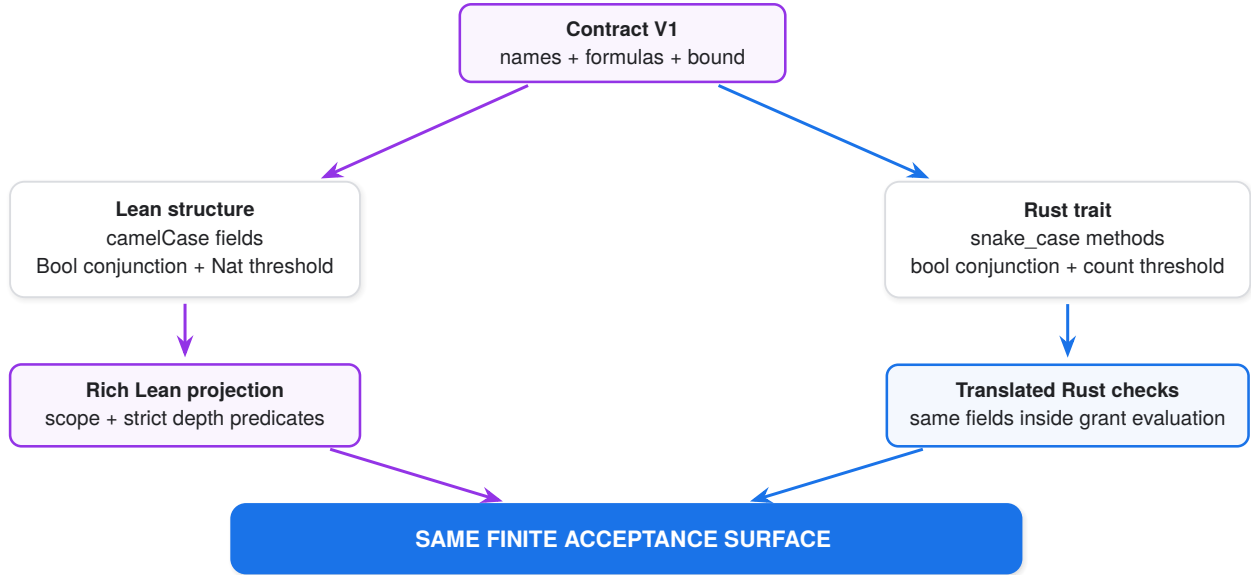


Figure 6: The retained finite algebra boundary. The contract prevents silent field drift. The rich Rust predicates feeding this boundary are now covered by the separate production refinement chain in Figure 5.

5. Executable refinement evidence

Proofs establish universal statements under their premises. The artifact also uses independent executable evidence to expose specification, translation, and wiring errors with concrete counterexamples.

5.1 Rich vectors and required mutations

Auths.VectorExport emits 23 byte-stable rich-authority vectors from Lean. They cover accepted and rejected scope attenuation, linked delegation, terminal action coverage, ordered diagnostics, exact profile behavior, set membership and subset, inclusive interval endpoints, action constraints, status freshness, assurance identity, and both meanings of an absent optional budget. Rust replays each vector through the shipping pure evaluators.

A checked-in 21-case mutation matrix names semantic mistakes that must be detected. It includes reversed interval and subset directions, negated membership, incorrect action-constraint constructors, budget direction and algebra confusion, conflating no requested spend with unbounded authority, weakened status/profile/assurance checks, non-strict depth, and partial principal or grant linkage. Each mutation has a witness, and the gate requires every mutation to be killed. This is a regression obligation, not a claim that mutation testing proves completeness.

5.2 Exhaustive finite algebra vectors

The same Lean executable remains the oracle for the generated finite algebra. It does not call a Rust reference evaluator. For attenuation it enumerates all assignments in $\{0, 1\}^{10}$:

$$2^{10} = 1,024 \text{ cases.}$$

Exactly one assignment is accepted because V1 aggregation is conjunction. For threshold counts, the declared exhaustive bound is $B = 16$. The exporter enumerates:

$$1 \leq k \leq B, \quad 0 \leq a \leq B, \quad 0 \leq u \leq B - a,$$

giving

$$B \sum_{a=0}^B (B - a + 1) = 16 \cdot 153 = 2,448$$

states. Rust checks every vector against `auths_algebra_kernel::threshold_counts`. It also constructs a real 16-leaf `AuthorizationPlan::k_of_n`, feeds the declared branch outcomes to shipping composition, and checks both the result and full leaf visitation.

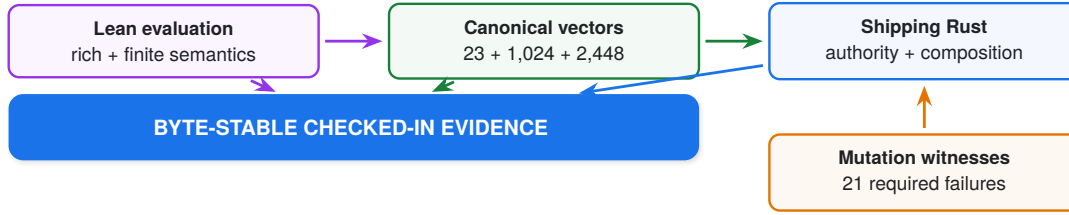


Figure 7: Executable evidence provenance. Lean supplies expected semantics; shipping Rust is the consumer. Named mutations require the evidence to distinguish security-relevant near misses.

5.3 Kani over shipping predicates

Kani lowers Rust MIR into CBMC’s bit-precise model-checking pipeline and checks assertions over symbolic inputs (Delmas et al. 2026; Kroening, Schrammel, and Tautschnig 2023). Four harnesses cover two crates:

- threshold classification under the declared bound;
- conjunction over all ten attenuation Booleans;
- the exact fixed-width inclusive-window relation; and
- three-window containment transitivity.

Kani checks reachable arithmetic and control flow in compiled Rust. It does not replace Lean’s unbounded mathematical proofs, and the Lean theorems do not replace Kani’s implementation-level checks.

5.4 One reproducible gate

`cargo xtask formal`:

1. validates the versioned algebra contract and rejects generated-source drift;
2. verifies the locked Rust, Lean, and Kani toolchains;
3. builds every Lean module;
4. audits 72 compiled statements, statement hashes, source closures, residual assumptions, and transitive axioms;
5. validates the checked-in Aeneas qualification, generated inventory, translation reports, reviewed links, warning inventory, and zero-axiom policy;
6. regenerates rich, attenuation, and threshold vectors and rejects byte drift;
7. replays vectors and the mutation matrix through shipping Rust; and
8. runs both Kani crates and all four harnesses.

The stronger qualification command re-extracts production source twice and requires byte-identical output:

```
cargo xtask formal qualify aeneas
```

Intentional artifact updates require the explicit `--update` mode. Ordinary validation is read-only.

5.5 Evaluation results

Artifact	Observed result	Established scope
Lean assurance manifest	72 / 72 compiled claims	56 rich/refinement and 16 composition statements, with reviewed axioms
Production refinement	3 / 3 theorems	author scope, linked delegation transition, and terminal coverage
Rich semantic vectors	23 / 23 replayed	production-shaped positive, negative, boundary, and diagnostic cases
Required mutations	21 / 21 killed	named semantic near misses have concrete distinguishing witnesses
Attenuation vectors	1,024 / 1,024 replayed	all Boolean assignments over ten generated predicates
Threshold and plan vectors	2,448 / 2,448 agreed	all valid counts and shipping ‘k-of-n’ evaluation through 16 leaves
Kani	4 / 4 harnesses; 0 failures	symbolic bounded algebra and interval predicates
Aeneas qualification	42 model + 1 algebra + 4 authority functions	no opaque local functions; 4 reviewed links; 0 compiled external axioms

Table 2: Formal artifact results at the evaluated revision. Counts describe semantic evidence, not performance or whole-system coverage.

6. Assurance boundary

6.1 What is proved

The Lean kernel checks:

- set, interval, action-constraint, budget, status, profile, and scope relation laws under the model’s canonical representation premises;
- downward closure of action coverage, evidence requirements, and admission;
- exact linked delegation acceptance, ordered diagnostics, root preservation, strict depth decrease, and uniqueness of the returned transition fields;
- exact agreement between three translated production evaluators and the rich author-scope, delegation, and terminal-coverage decisions;
- three-valued composition, threshold, monotonicity, traversal, diversity, termination, and declared structural-cost claims listed in the manifest.

The rich and composition theorems are not limited to the 16-leaf vector bound. Each compiled claim has a separately reviewed statement and scope; a count of theorems is not a substitute for reading those statements.

6.2 What is finite or bounded evidence

The threshold relation is exhaustively replayed through 16 leaves, equal to `DEFAULT_MAX_PLAN_LEAVES` for target V1, not through the configurable hard maximum of 128. The ten-Boolean attenuation interface is finite and completely enumerated. The 23 rich vectors and 21 mutation witnesses are targeted rather than exhaustive over rich values. Kani proves its four harness properties only over each harness’s symbolic bounds and assumptions.

These layers are deliberately labelled. Finite evidence catches boundary, serialization, and production-routing mistakes; it does not enlarge a Lean theorem’s premises or scope.

6.3 Trusted computing base and outer boundary

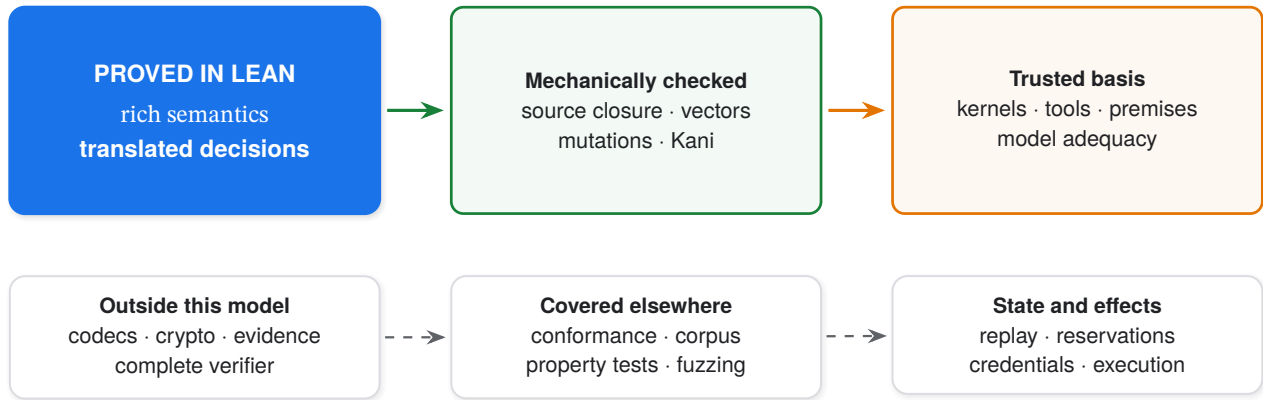


Figure 8: Claim and trust boundary. Dark blue is theorem-proved; green is mechanically cross-checked; amber is an explicit premise or tool. The lower row is outside this formal model.

The trusted computing base includes:

- the Lean kernel, imported libraries, and reviewed uses of `propext`, `Classical.choice`, and `Quot.sound`;
- Charon, Aeneas, the Rust compiler, Kani/CBMC, and the pinned build environment;
- the reviewed `String::as_bytes` external model and the faithful handling of validated constructors and borrowed views;
- the deterministic generator for the retained finite Boolean algebra;
- the adequacy of the rich model as the intended target-V1 authorization specification; and
- every component outside the isolated authority decisions, including codecs, cryptography, evidence, stores, adapters, credentials, and effects.

Rust’s type system and `unsafe-free` core reduce memory-safety risk, but Rust language safety is not itself a proof of authorization correctness. RustBelt illustrates why language safety and application semantics remain separate claims (Jung et al. 2018).

6.4 Residual representation and system boundary

The former gap between `Nat` coordinates in Lean and independently written Rust set, interval, constraint, budget, status, profile, and assurance relations is closed for the three translated evaluator decisions. What remains is a different boundary.

The theorems begin with validated production views. They do not prove that arbitrary bytes decode canonically, that every constructor preserves its invariant, that signatures establish control, or that acquired evidence is fresh and authentic. The mutable delegation caller also performs structural commit work around the translated result: it preserves private root and assurance state while cloning accepted transition fields. Those obligations are named in the manifest rather than overstated as translated facts.

Likewise, an authorization decision is not an external effect. Replay protection, durable reservations, budget accounting, credential release, crash reconciliation, and idempotent execution require separate state-machine semantics. Closing the rich projection gap makes those next layers possible; it does not make them unnecessary.

7. Security consequences

7.1 Delegation escalation

The product-order design makes widening explicit. A child must not:

- change the local root;
- keep or increase remaining depth;
- select a broader or different profile;
- add permissions or audiences;

- extend validity;
- broaden action-body discretion;
- remove or increase a bounded budget;
- weaken status freshness;
- weaken assurance.

The aggregate function cannot accidentally omit a declared dimension because the generated trait and generated conjunction share the contract's field list. More importantly, the translated production evaluator and rich refinement theorems now cover how every declared rich predicate is computed. The remaining risks sit at the validated-view boundary and outside the isolated evaluator, not in an independently trusted Boolean projection.

7.2 Fail-closed uncertainty

Three-valued threshold semantics prevent unavailable evidence from being treated as success. They also preserve enough information to distinguish a permanent denial from a potentially satisfiable missing fact. The threshold soundness theorems show:

- authorization requires at least k established branches;
- denial means even every indeterminate branch cannot reach k ;
- indeterminate means k is not established but remains reachable.

No diagnostic ordering can change that truth result. Canonical reason selection exists for deterministic reporting, not as an input to authorization.

7.3 Bounded work

Strictly decreasing depth gives a simple termination measure for delegation chains. Structural plan recursion terminates because plans are finite inductive values. Production additionally validates deployment limits before evaluation and reserves work before variable-cost adapters and cryptography.

The Lean theorem `evaluation_cost_linear_in_nodes` concerns the declared structural cost function. It should not be read as an empirical runtime complexity proof for the full verifier. Allocation, hashing, signature verification, adapter work, and codec behavior remain outside that theorem.

7.4 Change control as a security property

Authorization semantics evolve. The generated boundary is therefore valuable even if no current theorem fails. It changes the review shape:

SEMANTIC CHANGE RULE

A new truth value, threshold rule, or attenuation dimension must appear as one reviewable semantic change whose rich Lean relation, shipping pure Rust predicate, Aeneas source closure, refinement theorem, assurance-manifest statement, mutation witness, generated finite boundary, and checked vectors change together.

This is not merely build convenience. It prevents a class of silent specification drift in which a production check is added without a formal counterpart, a translated dependency becomes opaque, or a formal obligation is strengthened without changing shipping behavior.

8. What the core enables

The verified algebra is deliberately below transport and product policy. That placement lets outer packages vary without redefining authority.

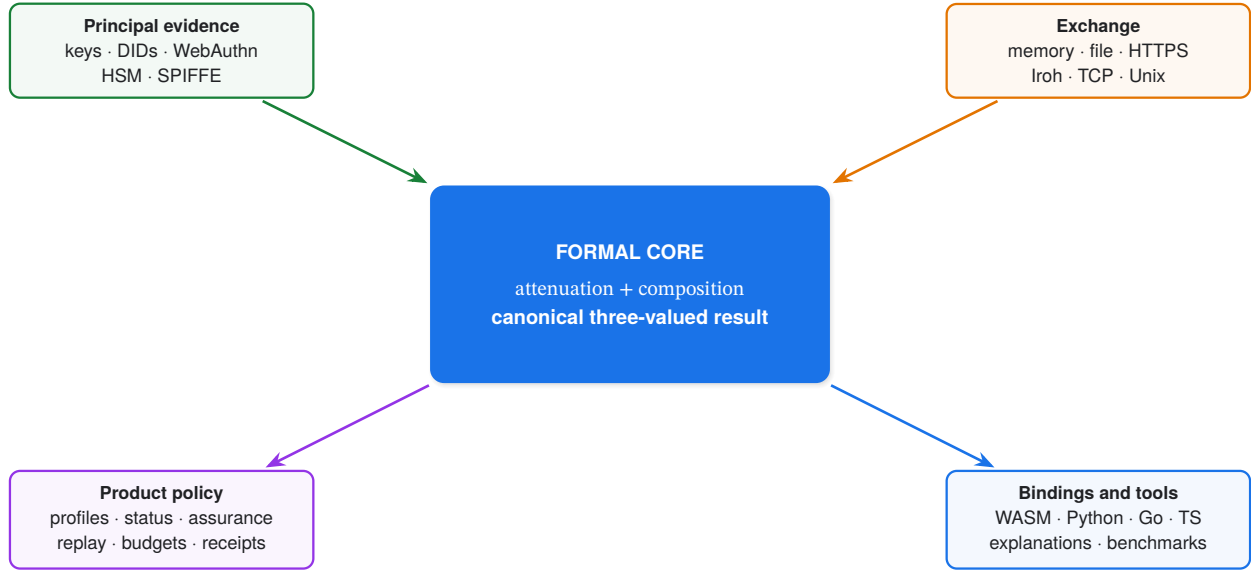


Figure 9: The algebra as a narrow semantic center. Outer components supply evidence, carry bytes, enforce state, and expose APIs; they do not redefine attenuation or composition.

Seven principal methods can produce typed control evidence for the same verifier. Exchange transports carry opaque proof bytes but cannot upgrade an invalid proof. Product packages can add atomic replay and budget stores, receipts, custody, profiles, and enforcement. Native and WASM boundaries can consume the same canonical result. Deployment explanations can report which trusted-context facts caused or prevented authorization.

These packages demonstrate architectural leverage, not additional formal coverage. A correct algebra cannot compensate for an adapter that overstates control, a profile that assigns unsafe meaning, a stale status source, or an executor that ignores the sealed action.

The GitHub, Radicle, Stripe, Kubernetes, OpenTofu, and PostgreSQL integrations also show why the rich core is not the final abstraction. Real domains require bounded policy over quantities, time, resources, and predicates, followed by durable reservation and exact execution. Those shared stateful semantics should be derived from multiple complete domains and formalized above this authority kernel, rather than pushed into an open-ended core policy language.

9. Related work

9.1 Authorization and attenuation

Capability systems made authority an explicit transferable object (Dennis and Horn 1966). SPKI authorization certificates bind permissions to keys and support delegation (Ellison et al. 1999). SDSI emphasizes linked local namespaces (Rivest and Lampson 1996). Macaroons support efficient contextual attenuation through caveats (Birgisson et al. 2014). Trust-management systems such as PolicyMaker, KeyNote, and RT separate assertions, local policy, and compliance checking (Blaze, Feigenbaum, and Lacy 1996; Blaze et al. 1999; Li, Mitchell, and Winsborough 2002).

Auths-Proof shares explicit delegation and local policy but uses a closed multidimensional order rather than a general policy language. That restriction makes the core algebra finite at its production projection boundary and tractable for exhaustive cross-language checking.

Proof-carrying code shifts the burden of supplying safety evidence to the producer (Necula 1997). Proof-carrying authentication applies a related idea to distributed authorization judgments (Appel and Felten 1999; Bauer, Schneider, and Felten 2002). Auths-Proof authorization proofs are protocol evidence, not Lean proof terms; the receiver still executes a fixed verifier.

9.2 Formal refinement

CompCert proves semantic preservation through a realistic compiler (Leroy 2009). seL4 proves refinement from an abstract specification to a C implementation (Klein et al. 2009). Translation validation checks a particular translation result (Pnueli, Siegel, and Singerman 1998). Auths-Proof’s generated boundary is a small translation-validation arti-

fact for the finite algebra. Its rich authority path instead translates exact production functions and proves functional refinement under representation premises; neither Charon nor Aeneas is itself proved correct by this artifact.

Lean 4 supplies the theorem-proving environment (Moura and Ullrich 2021). Aeneas uses functional translation to make safe Rust amenable to proof assistants (Ho and Protzenko 2022). Verus provides a practical Rust-centered systems-verification environment (Lattuada et al. 2024). Auths-Proof uses Aeneas because its generated functional definitions can be related directly to the handwritten Lean model, while retaining Kani as an independent MIR-level check.

9.3 Model checking and testing

Kani and CBMC provide bit-precise bounded model checking over the shipping Rust function (Delmas et al. 2026; Kroening, Schrammel, and Tautschnig 2023). This complements, rather than duplicates, Lean: Kani sees Rust control flow and machine arithmetic, while Lean proves unbounded properties of the mathematical definition.

Property-based testing and fuzzing remain valuable at the richer boundaries that the Lean model excludes (Claessen and Hughes 2000; Miller, Fredriksen, and So 1990). They search large input spaces and preserve counterexamples, but passing campaigns do not establish universal properties. The artifact therefore labels theorem, exhaustive enumeration, bounded model checking, conformance testing, and fuzzing separately.

10. Limitations and future work

This paper does not claim full functional correctness of the verifier. It establishes rich authority semantics, three production-function refinement theorems under representation premises, an unbounded composition algebra, exhaustive finite agreement for declared domains, and bounded checks of four shipping Rust properties.

The most important limitations are:

Model adequacy. A correct proof of the wrong model is still wrong. Review of the chosen authority dimensions and their order remains essential.

Representation and codec boundary. Refinement starts from validated production views. Canonical decoding, constructor preservation, view losslessness, and byte-to-rich abstraction are not end-to-end theorems.

Bound asymmetry. Lean threshold theorems are unbounded. Rust vector replay and the Kani threshold harness stop at 16, not the configurable hard maximum of 128.

Caller and whole-verifier exclusion. Structural state commit, canonical parsing, graph resolution, signature verification, evidence adapters, status acquisition, replay, final sealing, and receipt persistence are not refined to Lean.

Effectful bounded authorization. Optional ceilings in authority are not a formal model of concurrent reservations or aggregate spend. Bounded product policy, durable execution intent, crashes, unknown outcomes, reconciliation, and external side effects remain future layers.

Toolchain trust. Lean, Mathlib, Charon, Aeneas, Rust, Kani, CBMC, the generator, the build environment, and their dependencies remain part of the evidence pipeline.

The next work should proceed in decreasing order of semantic leverage:

1. connect canonical decoding, constructor invariants, and borrowed-view losslessness to the rich representation bridge;
2. translate or otherwise refine the structural caller that commits accepted delegation fields and preserves private root and assurance state;
3. derive a closed bounded-policy algebra from six end-to-end domains, then prove tightening, arithmetic, freshness, reservation, replay, crash, and reconciliation properties;
4. refine credential release and exact external execution to the authorized action and durable intent;
5. extend symbolic threshold checks to the hard bound or prove the Rust classifier independently of enumeration; and
6. produce verifiable build attestations for source closure, generated artifacts, theorem statements, and pinned translation tools.

11. Conclusion

Formalization is valuable only to the extent that its relationship to shipping behavior is understood. Auths-Proof no longer stops at proving a conjunction of predicates produced elsewhere. It models the rich authority relations, reshapes shipping decisions into isolated pure Rust, translates those exact functions, proves their authoring, delegation, and terminal-coverage results against the rich model, and audits the entire evidence chain as a release artifact.

The resulting claim is intentionally precise:

Under published validated-representation and toolchain assumptions, Auths-Proof’s exact isolated production Rust evaluators for authoring, delegation, and terminal coverage refine its rich target-V1 Lean semantics; the generated composition boundary separately agrees over its declared finite domain while Lean proves its central algebraic properties without that bound.

That claim is narrower than whole-program verification and substantially stronger than parallel handwritten models. The former rich-projection gap is closed at the shipping decision boundary. The remaining work is now clearer: bytes to validated views, structural commit, cryptography and evidence, stateful bounded authorization, credentials, external effects, and complete verifier control flow. The system can improve along a visible refinement ladder rather than accumulating informal confidence around an isolated proof artifact.

References

- Abadi, Martín, Michael Burrows, Butler Lampson, and Gordon Plotkin. 1993. “A Calculus for Access Control in Distributed Systems.” *ACM Transactions on Programming Languages and Systems* 15 (4): 706–34. <https://doi.org/10.1145/155183.155225>.
- Appel, Andrew W., and Edward W. Felten. 1999. “Proof-Carrying Authentication.” In *Proceedings of the 6th ACM Conference on Computer and Communications Security*, 52–62. <https://doi.org/10.1145/319709.319718>.
- Bauer, Lujo, Michael A. Schneider, and Edward W. Felten. 2002. “A General and Flexible Access-Control System for the Web.” In *Proceedings of the 11th USENIX Security Symposium*, 93–108. USENIX Association. https://www.usenix.org/legacy/events/sec02/full_papers/bauer/bauer_html/.
- Birgisson, Arnar, Joe Gibbs Politz, Úlfar Erlingsson, Ankur Taly, Michael Vrabie, and Mark Lentzner. 2014. “Macaroons: Cookies with Contextual Caveats for Decentralized Authorization in the Cloud.” In *Network and Distributed System Security Symposium*. Internet Society. <https://research.google/pubs/macaroons-cookies-with-contextual-caveats-for-decentralized-authorization-in-the-cloud/>.
- Blaze, Matt, Joan Feigenbaum, John Ioannidis, and Angelos D. Keromytis. 1999. “The KeyNote Trust-Management System Version 2.” RFC 2704. RFC Editor. <https://doi.org/10.17487/RFC2704>.
- Blaze, Matt, Joan Feigenbaum, and Jack Lacy. 1996. “Decentralized Trust Management.” In *Proceedings of the 1996 IEEE Symposium on Security and Privacy*, 164–73. <https://doi.org/10.1109/SECPR1.1996.502679>.
- Burrows, Michael, Martín Abadi, and Roger Needham. 1990. “A Logic of Authentication.” *ACM Transactions on Computer Systems* 8 (1): 18–36. <https://doi.org/10.1145/77648.77649>.
- Claessen, Koen, and John Hughes. 2000. “QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs.” In *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming*, 268–79. <https://doi.org/10.1145/351240.351266>.
- Delmas, Rémi, Ziyad Hassan, Qinheping Hu, Rahul Kumar, Felipe R. Monteiro, Thanh Nguyen, Adrián Palacios, et al. 2026. “Kani: A Model Checker for Rust.” *arXiv Preprint arXiv:2607.01504*. <https://doi.org/10.48550/arXiv.2607.01504>.
- Dennis, Jack B., and Earl C. Van Horn. 1966. “Programming Semantics for Multiprogrammed Computations.” *Communications of the ACM* 9 (3): 143–55. <https://doi.org/10.1145/365230.365252>.
- Ellison, Carl M., Bill Frantz, Butler Lampson, Ron Rivest, Brian M. Thomas, and Tatu Ylonen. 1999. “SPKI Certificate Theory.” RFC 2693. RFC Editor. <https://doi.org/10.17487/RFC2693>.
- Ho, Son, and Jonathan Protzenko. 2022. “Aeneas: Rust Verification by Functional Translation.” *Proceedings of the ACM on Programming Languages* 6 (ICFP): 711–41. <https://doi.org/10.1145/3547647>.
- Jung, Ralf, Jacques-Henri Jourdan, Robbert Krebbers, and Derek Dreyer. 2018. “RustBelt: Securing the Foundations of the Rust Programming Language.” *Proceedings of the ACM on Programming Languages* 2 (POPL): 66:1–34. <https://doi.org/10.1145/3158154>.
- Klein, Gerwin, Kevin Elphinstone, Gernot Heiser, June Andronick, David Cock, Philip Derrin, Dhammika Elkaduwe, et al. 2009. “seL4: Formal Verification of an OS Kernel.” In *Proceedings of the 22nd ACM Symposium on Operating Systems Principles*, 207–20. <https://doi.org/10.1145/1629575.1629596>.
- Kroening, Daniel, Peter Schrammel, and Michael Tautschnig. 2023. “CBMC: The C Bounded Model Checker.” *arXiv Preprint arXiv:2302.02384*. <https://doi.org/10.48550/arXiv.2302.02384>.
- Lattuada, Andrea, Travis Hance, Jay Bosamiya, Matthias Brun, Chanhee Cho, Hayley LeBlanc, Pranav Srinivasan, et al. 2024. “Verus: A Practical Foundation for Systems Verification.” In *Proceedings of the 30th ACM Symposium on Operating Systems Principles*, 438–54. <https://doi.org/10.1145/3694715.3695952>.
- Leroy, Xavier. 2009. “Formal Verification of a Realistic Compiler.” *Communications of the ACM* 52 (7): 107–15. <https://doi.org/10.1145/1538788.1538814>.
- Li, Ninghui, John C. Mitchell, and William H. Winsborough. 2002. “Design of a Role-Based Trust-Management Framework.” In *Proceedings of the 2002 IEEE Symposium on Security and Privacy*, 114–30. <https://doi.org/10.1109/SECPR1.2002.1004376>.
- Miller, Barton P., Lars Fredriksen, and Bryan So. 1990. “An Empirical Study of the Reliability of UNIX Utilities.” *Communications of the ACM* 33 (12): 32–44. <https://doi.org/10.1145/96267.96279>.
- Moura, Leonardo de, and Sebastian Ullrich. 2021. “The Lean 4 Theorem Prover and Programming Language.” In *Automated Deduction – CADE 28*, 12699:625–35. Lecture Notes in Computer Science. Springer. https://doi.org/10.1007/978-3-030-79876-5_37.
- Necula, George C. 1997. “Proof-Carrying Code.” In *Proceedings of the*

- 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, 106–19.
<https://doi.org/10.1145/263699.263712>.
- Pnueli, Amir, Michael Siegel, and Eli Singerman. 1998. “Translation Validation.” In *Tools and Algorithms for the Construction and Analysis of Systems*, 1384:151–66. Lecture Notes in Computer Science. Springer. <https://doi.org/10.1007/BFb0054170>.
- Rivest, Ronald L., and Butler Lampson. 1996. “SDSI: A Simple Distributed Security Infrastructure.” Technical memo. Massachusetts Institute of Technology.
<https://www.microsoft.com/en-us/research/publication/sdsi-a-simple-distributed-security-infrastructure/>.